Optimisation dans les Réseaux

Graphes, Problèmes de Base et Connexité

Cours détaillé avec exemples et notes complémentaires

F. Bendali-Mailfert

Version 2025 - Enrichie

Contents

1 Introduction **4**

2 Problèmes de Réseaux et Structures de Modélisation **5**
2.1 Principaux problèmes dans les réseaux 5

3 Rappels de Graphes : Définitions et Notations **6**
3.1 Graphes non orientés et orientés 6
3.1.1 Définition d'un graphe 6
3.1.2 Exemple illustratif 7
3.2 Incidence et Adjacence 8
3.2.1 Définitions 8
3.2.2 Matrices de représentation - Cas non orienté 8
3.2.3 Matrices de représentation - Cas orienté 9
3.2.4 Exemple de matrices 10
3.3 Degré et Graphe Complet 11
3.3.1 Degré d'un sommet 11
3.3.2 Graphe complet 11
3.4 Sous-graphes 12
3.4.1 Définitions 12
3.4.2 Stables et graphes k-partis 13
3.5 Cheminements 14
3.5.1 Chaînes et cycles (graphes non orientés) 14
3.5.2 Chemins et circuits (graphes orientés) 15
3.6 Connexité 16
3.6.1 Définitions 16
3.6.2 Coupe (Cocycle) 16
3.7 Arbres 17
3.7.1 Définition et propriétés 17

4 Réseaux : Définitions et Notations **19**
4.1 Notion de réseau 19
4.2 Notations et paramètres 20

5 Complexité d'un Algorithme **21**
5.1 Définition 21

6 Problèmes de Base dans les Réseaux **22**
6.1 Problème du Plus Court Chemin 22
6.1.1 Description du problème 22
6.1.2 Formulation linéaire 23
6.1.3 Remarques importantes 23
6.1.4 Algorithmes de résolution 24
6.2 Problème du Flot Maximum 25
6.2.1 Description du problème 25
6.2.2 Formulation linéaire 26
6.2.3 Algorithmes de résolution 27
6.3 Problème du Flot de Coût Minimum 29
6.3.1 Description du problème 29
6.3.2 Formulation linéaire 30
6.3.3 Algorithmes de résolution 31

7 Connexité dans les Réseaux de Télécommunications **33**
7.1 Problèmes généraux sur la connexité 33
7.2 Architecture des Réseaux de Télécommunications 34
7.2.1 Les trois niveaux de traitement 34
7.3 Problèmes de Longue Distance 35
7.3.1 Le long terme (> 10 années) 35
7.3.2 Le moyen terme (< 5 années) 36
7.4 Solutions pour les Longues Distances 37
7.4.1 Solution de base : les arbres 37
7.4.2 Modèle linéaire pour l'arbre de poids minimum 37
7.5 Réseaux k-Connexes 39
7.5.1 Définition 39
7.5.2 Types de nuds selon les requêtes 40
7.5.3 Contraintes de robustesse 40
7.6 Cas Particuliers et Complexité 41
7.7 Approche de Résolution : Réseau 2-Connexe 42
7.7.1 Modèle linéaire entier pour $r_i = 2, \forall i \in V$ 42
7.7.2 Méthode de résolution : Branch & Cut 42
7.8 Théorème de Menger (1927) 44
7.8.1 Énoncé pour les réseaux (version arcs) 44
7.8.2 Énoncé pour les graphes non orientés 44
7.8.3 Conséquence : Théorème de Whitney (1932) 45
7.9 Exemple d'Application : Réseau Belge Belgacom 46
7.9.1 Contexte 46
7.9.2 Solution proposée 46

8 Résumé et Points Essentiels **47**
8.1 Structures de graphes 47
8.2 Problèmes d'optimisation 47
8.3 Connexité et robustesse 48
8.4 Applications pratiques 48

9 Conclusion **49**

10 Structures et Algorithmes dans les Réseaux Sans Fil **64**
10.1 Généralités sur les Réseaux Sans Fil 64
10.1.1 Réseaux multisauts 64
10.1.2 Structure du réseau virtuel 65
10.2 Découverte de Voisinage et Élimination 66
10.2.1 Filtrage du voisinage 66
10.3 Contrôle de Topologie 67
10.3.1 Types d'algorithmes 67
10.3.2 Types de contrôle de l'énergie 67
10.4 Algorithmes Centralisés 68
10.4.1 Algorithme 1 : Arbre couvrant minimum (MST) 68
10.4.2 Algorithme 2 : BIP (Broadcast Incremental Power) 69
10.4.3 BIP Adapté (Version Bidirectionnelle) 70
10.5 Algorithmes Locaux 71
10.5.1 LMST (Local Minimum Spanning Tree) 71
10.5.2 RNG (Relative Neighborhood Graph) 72
10.6 Structuration du Réseau : Organisation des Échanges 73
10.6.1 Protocoles de routage 73
10.7 Dominants et Clustering 74
10.7.1 Définitions 74
10.7.2 Clustering (Partitionnement en groupes) 75
10.8 Algorithmes de Diffusion 77
10.9 Synthèse : Réseaux Sans Fil 78

1 Introduction

Note de Cours

Ce cours présente les fondements de l'optimisation dans les réseaux, un domaine essentiel de la recherche opérationnelle. Nous étudierons les structures de graphes, les problèmes classiques (plus court chemin, flot maximum, flot de coût minimum) et les questions de connexité dans les réseaux de télécommunications.

Important

L'optimisation dans les réseaux trouve des applications concrètes dans :

- **Les télécommunications** : topologie et routage
- **Les transports** : logistique et distribution
- **L'informatique** : réseaux informatiques et flux de données
- **L'énergie** : distribution électrique et gaz

2 Problèmes de Réseaux et Structures de Modélisation

2.1 Principaux problèmes dans les réseaux

Définition

Les problèmes fondamentaux dans les réseaux sont :
1. Le dimensionnement : Rechercher les capacités à mettre sur les liaisons dans un réseau.
2. La topologie (design) : Rechercher la structure optimale : nuds principaux et liens.
3. Le routage : Rechercher les "meilleurs" chemins pour acheminer le trafic.

Note de Cours

Domaines d'application principaux :
Télécommunications :

- Topologie et routage dans les réseaux filaires
- Topologie dans les réseaux sans fil

Ces problèmes sont au cur de la conception et de l'exploitation des réseaux modernes.

3 Rappels de Graphes : Définitions et Notations

3.1 Graphes non orientés et orientés

3.1.1 Définition d'un graphe

Définition

Un **graphe** $G = (V, E)$ est un couple formé de deux ensembles disjoints :
Graphe non orienté :

- $V = \{v_1, v_2, \ldots, v_n\}$: ensemble de **sommets**
- $E = \{e_1, e_2, \ldots, e_m\}$: ensemble d'**arêtes**
- Pour tout i, e_i est une *paire* d'éléments de V

Graphe orienté :

- $V = \{v_1, v_2, \ldots, v_n\}$: ensemble de **sommets**
- $E = \{e_1, e_2, \ldots, e_m\}$: ensemble d'**arcs**
- Pour tout i, e_i est un *couple* d'éléments de V

Note de Cours

Notations importantes :
Pour un graphe non orienté :

- $u \in V$ et $v \in V$ sont les **extrémités** de l'arête $e = \{u, v\}$ ou $e = uv$

Pour un graphe orienté :

- $I(e) = u \in V$: extrémité **initiale** de l'arc $e = (u, v)$
- $T(e) = v \in V$: extrémité **terminale** de l'arc $e = (u, v)$ ou $\overrightarrow{uv}$

À savoir

Un **graphe simple** n'a ni boucle (arête reliant un sommet à lui-même) ni arête multiple (plusieurs arêtes entre les mêmes sommets).

3.1.2 Exemple illustratif

Exemple

Considérons un graphe $G = (V, E)$ avec :

- $V = \{1, 2, 3, 4, 5, 6\}$
- $E = \{(1, 2), (1, 3), (2, 3), (2, 4), (3, 4), (3, 5), (4, 5), (5, 6)\}$

Ce graphe peut représenter un réseau de communication où :

- Les sommets sont des routeurs ou des commutateurs
- Les arêtes/arcs sont des liens de communication

3.2 Incidence et Adjacence

3.2.1 Définitions

Définition

Incidence : Un sommet $v \in V$ et une arête $e \in E$ sont **incidents** si v est une extrémité de e.
Adjacence :

- Deux **sommets** sont adjacents lorsqu'ils sont incidents à une même arête
- Deux **arêtes ou arcs** sont adjacents lorsqu'ils sont incidents à un même sommet

3.2.2 Matrices de représentation - Cas non orienté

Définition

Matrice d'adjacence $A = (a_{ij})$:

$$a_{ij} = \text{nombre d'arêtes incidentes à } i \text{ et } j$$

Propriétés :

- A est **symétrique**
- A est à valeurs dans $\{0, 1\}$ pour un graphe simple

Matrice d'incidence $B = (b_{ij})$:

- Ligne i : sommet v_i
- Colonne j : arête e_j

$$b_{ij} = \text{nombre de fois où } v_i \text{ est incidente à } e_j$$

Propriétés :

- B est à valeurs dans $\{0, 1, 2\}$
- $\sum_{i=1}^{n} b_{ij} = 2$ pour tout j (chaque arête a deux extrémités)

3.2.3 Matrices de représentation - Cas orienté

Définition

Matrice d'adjacence : identique au cas non orienté (mais non symétrique)
Matrice d'incidence $B = (b_{ij})$:

$$b_{ij} = \begin{cases} +1 & \text{si } v_i = I(e_j) \text{ (extrémité initiale)} \\ -1 & \text{si } v_i = T(e_j) \text{ (extrémité terminale)} \\ 0 & \text{sinon} \end{cases}$$

Propriétés :

- B est à valeurs dans $\{-1, 0, +1\}$
- $\sum_{i=1}^{n} b_{ij} = 0$ pour tout j (conservation du flot)

Note de Cours

Convention de signe :

- $+1$: le sommet est le **départ** de l'arc (flux sortant)
- -1 : le sommet est l'**arrivée** de l'arc (flux entrant)
- 0 : le sommet n'est pas incident à l'arc

3.2.4 Exemple de matrices

Exemple

Soit un graphe non orienté avec 4 sommets et 6 arêtes $\{a, b, c, d, e, f\}$:
Matrice d'adjacence :

$$A = \begin{pmatrix} 0 & 1 & 2 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}$$

Interprétation : Il y a 2 arêtes entre les sommets 1 et 3.
Matrice d'incidence (non orienté) :

$$B = \begin{pmatrix} & a & b & c & d & e & f \\ 1 & 1 & 0 & 1 & 1 & 1 & 0 \\ 2 & 1 & 1 & 0 & 0 & 0 & 0 \\ 3 & 0 & 1 & 1 & 1 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & 1 & 2 \end{pmatrix}$$

Matrice d'incidence (orienté) :

$$B = \begin{pmatrix} & a & b & c & d & e & f \\ 1 & -1 & 0 & 1 & -1 & 1 & 0 \\ 2 & 1 & 1 & 0 & 0 & 0 & 0 \\ 3 & 0 & -1 & -1 & 1 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & -1 & 0 \end{pmatrix}$$

Note : La somme de chaque colonne est 0.

3.3 Degré et Graphe Complet

3.3.1 Degré d'un sommet

Définition

Le **degré** $d(v)$ d'un sommet v est le nombre d'arêtes incidentes à ce sommet.
Notations :

- $\delta(G) = \min\{d(v), v \in V\}$: degré **minimum**
- $\Delta(G) = \max\{d(v), v \in V\}$: degré **maximum**

À savoir

Propriété importante : La somme des degrés de tous les sommets d'un graphe est égale à deux fois le nombre d'arêtes :

$$\sum_{v \in V} d(v) = 2|E|$$

Conséquence : Le nombre de sommets de degré impair est toujours pair.

3.3.2 Graphe complet

Définition

Un **graphe complet** (ou **clique**) est un graphe où deux sommets quelconques sont adjacents.
Notation : K_n pour un graphe complet à n sommets.
Propriétés :

- Nombre d'arêtes : $|E| = \frac{n(n-1)}{2}$
- Degré de chaque sommet : $d(v) = n - 1$ pour tout $v \in V$
- $\delta(K_n) = \Delta(K_n) = n - 1$

Exemple

Graphes complets :

- K_3 (triangle) : 3 sommets, 3 arêtes
- K_4 (tétraèdre) : 4 sommets, 6 arêtes
- K_5 : 5 sommets, 10 arêtes
- K_n : n sommets, $\frac{n(n-1)}{2}$ arêtes

3.4 Sous-graphes

3.4.1 Définitions

Définition

Soit $G = (V, E)$ un graphe, $U \subseteq V$ et $F \subseteq E$.

1. Sous-graphe général :
$H = (U, F)$ est un **sous-graphe** de G si pour tout $e \in F$, ses extrémités sont dans U.

2. Sous-graphe engendré par des sommets :
$G_U = (U, F)$ est le sous-graphe de G **engendré par** U si F est formé par toutes les arêtes dont les extrémités sont dans U.

3. Sous-graphe engendré par des arêtes :
$G[F] = (U, F)$ est le sous-graphe **engendré par** F dont :

- Les arêtes sont dans F
- Les sommets sont formés des extrémités des arêtes de F

4. Graphe partiel :
Un sous-graphe $H = (U, F)$ est un **graphe partiel** si $U = V$ (tous les sommets sont conservés).

Note de Cours

Différence importante :

- **Sous-graphe engendré par** U : on prend TOUTES les arêtes entre sommets de U
- **Sous-graphe général** : on peut choisir seulement certaines arêtes
- **Graphe partiel** : on garde tous les sommets, on enlève des arêtes

3.4.2 Stables et graphes k-partis

Définition

Stable :
Un sous-ensemble S de sommets est un **stable** si le sous-graphe G_S engendré par S est vide d'arêtes (aucun sommet de S n'est adjacent à un autre sommet de S).
Graphe k-parti :
Un graphe est **k-parti** si son ensemble de sommets admet une partition en k stables.
Graphe biparti :
Un graphe est **biparti** s'il est 2-parti. On peut partitionner ses sommets en deux stables V_1 et V_2 tels que toute arête relie un sommet de V_1 à un sommet de V_2.

À savoir

Propriété caractéristique : Un graphe est biparti si et seulement s'il ne contient pas de cycle de longueur impaire.

Exemple

Exemples de graphes bipartis :

- Les arbres sont toujours bipartis
- Le graphe complet biparti $K_{m,n}$: m sommets dans un stable, n dans l'autre
- Les cycles de longueur paire

Non-biparti :

- Triangle (K_3)
- Tout cycle de longueur impaire

3.5 Cheminements

3.5.1 Chaînes et cycles (graphes non orientés)

Définition

Chaîne :
Une **chaîne** d'un graphe $G = (V, E)$ est une suite alternée de sommets et d'arêtes :

$$v_0 e_1 v_1 e_2 \dots v_{k-1} e_k v_k$$

telle que pour tout i, les extrémités de e_i sont v_{i-1} et v_i.
Chaîne simple :
Si toutes les **arêtes** sont différentes.
Chaîne élémentaire :
Si tous les **sommets** sont différents (implique que la chaîne est simple).
Cycle :
Un **cycle** est une chaîne *fermée* simple (où $v_0 = v_k$).
Longueur :
La longueur d'une chaîne ou d'un cycle est le nombre d'arêtes qui la compose.

Exemple

Considérons un graphe avec sommets $\{1, 2, 3, 4, 5, 6\}$ et arêtes $\{a, b, c, d, e, f, g, h, i, j\}$:
Chaîne simple : $1a2c3h6i2b3d4$ de longueur 6

- Toutes les arêtes sont différentes
- Le sommet 2 et le sommet 3 apparaissent deux fois

Chaîne élémentaire : $1j6h3f5e4$ de longueur 4

- Tous les sommets sont différents
- Donc toutes les arêtes sont aussi différentes

Cycle : $2c3d4e5f3b2$ de longueur 5

- Commence et finit au sommet 2
- Toutes les arêtes sont différentes

Cycle élémentaire : $1j6g5f3c2a1$ de longueur 5

- Tous les sommets intermédiaires sont différents
- Seul le sommet de départ/arrivée (1) est répété

3.5.2 Chemins et circuits (graphes orientés)

Définition

Chemin :
Un **chemin** de $G = (V, E)$ est une suite alternée de sommets et d'arcs :

$$v_0 e_1 v_1 e_2 \dots v_{k-1} e_k v_k$$

telle que pour tout i, l'arc e_i admet v_{i-1} comme extrémité initiale et v_i comme extrémité terminale.
Chemin simple :
Si tous les **arcs** sont différents.
Chemin élémentaire :
Si tous les **sommets** sont différents.
Circuit :
Un **circuit** est un chemin *fermé* simple.

Note de Cours

Différence fondamentale avec les graphes non orientés :
Dans un graphe orienté, on doit respecter le sens des arcs. Un arc (u, v) ne peut être emprunté que de u vers v, pas dans l'autre sens.

Exemple

Dans un graphe orienté :
Chemin élémentaire : $1 \to 6 \to 2 \to 5 \to 3 \to 4$ de longueur 5

- Tous les sommets sont différents
- On suit le sens des arcs

Chaîne élémentaire : $1 \to 2 \to 3 \to 5 \to 4$ de longueur 4

- On peut ignorer le sens des arcs (considérer le graphe sous-jacent)

Circuit : $2 \to 5 \to 3 \to 2$ de longueur 3

- Commence et finit au même sommet
- Respecte le sens des arcs

Cycle : $3 \to 4 \to 5 \to 3$ de longueur 3

- Peut être vu comme un circuit dans le graphe non orienté sous-jacent

3.6 Connexité

3.6.1 Définitions

Définition

Connexité :
Un graphe est **connexe** si et seulement si entre tout couple de sommets existe une chaîne.
Forte connexité :
Un graphe orienté est **fortement connexe** si et seulement si entre tout couple de sommets existe un chemin (dans les deux sens).

À savoir

Vérification pratique :

- Pour un graphe **non orienté** : il suffit qu'il n'y ait qu'une seule composante connexe
- Pour un graphe **orienté** : depuis chaque sommet, on doit pouvoir atteindre tous les autres en suivant les arcs

3.6.2 Coupe (Cocycle)

Définition

Une **coupe** (ou **cocycle**) associée à un ensemble de sommets $S \subseteq V$ dans un graphe $G = (V, E)$ est un sous-ensemble d'arêtes dont une extrémité exactement est dans S.
Notation :

$$[S, V \setminus S] = \delta(S) = \{e = uv \in E : u \in S, v \in V \setminus S\}$$

Note de Cours

Interprétation :
La coupe $\delta(S)$ représente l'ensemble des arêtes qui "traversent" la frontière entre S et son complémentaire $V \setminus S$.
Si on supprime toutes les arêtes de $\delta(S)$, le graphe est déconnecté en (au moins) deux composantes : S et $V \setminus S$.

Exemple

Soit $V = \{1, 2, 3, 4, 5, 6\}$ et $S = \{1, 2, 3\}$.
Si les arêtes sont : $\{(1, 2), (2, 3), (1, 3), (3, 4), (2, 5), (4, 5), (5, 6)\}$
Alors : $\delta(S) = \{(3, 4), (2, 5)\}$
Ces deux arêtes sont les seules qui relient S au reste du graphe.

3.7 Arbres

3.7.1 Définition et propriétés

Définition

Un **arbre** $A = (V, E)$ est un graphe connexe sans cycle.

Important

Un arbre satisfait les propriétés équivalentes suivantes :

1. $|E| = |V| - 1$ (nombre d'arêtes = nombre de sommets − 1)
2. Il existe **exactement une chaîne** entre tout couple de sommets
3. C'est un graphe **connexe minimal** : supprimer n'importe quelle arête le déconnecte
4. C'est un graphe **sans cycle maximal** : ajouter n'importe quelle arête crée un cycle

Note de Cours

Terminologie pour les arbres :

- **Racine** : sommet distingué dans un arbre enraciné
- **Feuille** : sommet de degré 1
- **Nud interne** : sommet qui n'est pas une feuille
- **Profondeur** : distance depuis la racine
- **Hauteur** : profondeur maximale

Exemple

Exemple d'arbre :
Soit un arbre à 6 sommets $\{1, 2, 3, 4, 5, 6\}$ avec arêtes $\{(1, 2), (1, 3), (2, 4), (2, 5), (3, 6)\}$
Propriétés vérifiées :

- $|E| = 5 = 6 - 1 = |V| - 1$ (verifie)
- Connexe : on peut aller de n'importe quel sommet à n'importe quel autre (verifie)
- Sans cycle : aucune chaîne fermée (verifie)
- Entre 1 et 6 : une seule chaîne $1 - 3 - 6$ (verifie)

Feuilles : sommets 4, 5, 6 (degré 1)

À savoir

Applications des arbres :

- Arbres de recouvrement de coût minimum (MST)
- Arbres de décision
- Hiérarchies et taxonomies
- Structures de données (arbres binaires, B-arbres)
- Réseaux minimaux de télécommunication

4 Réseaux : Définitions et Notations

4.1 Notion de réseau

Définition

Un **réseau** est un graphe orienté $G = (V, E)$ dont les arcs ou les nuds sont munis d'une ou plusieurs valeurs (coûts, capacités, demandes).

4.2 Notations et paramètres

Définition

Dans la suite, nous utiliserons les notations suivantes :
Pour les sommets ($\forall i \in V$) :

- $b_i < 0$: le sommet i a une **demande** (puits, destination)
- $b_i > 0$: le sommet i a une **offre** (source, origine)
- $b_i = 0$: le sommet i est un **nud de transit** (intermédiaire)

Pour les arcs ($\forall (i,j) \in E$) :

- c_{ij} : **coût unitaire** pour traverser l'arc (i,j) (notation : $c_{ij} < C$)
- u_{ij} : **capacité supérieure** de l'arc (i,j) (notation : $u_{ij} < U$)
- l_{ij} : **capacité inférieure** de l'arc (i,j) (notation : $l_{ij} < L$)

Note de Cours

Conservation du flot :
Dans un réseau de flot, on a généralement la contrainte :

$$\sum_{i \in V} b_i = 0$$

C'est-à-dire que la somme totale des offres égale la somme totale des demandes. Ce qui entre dans le réseau doit en sortir.

Exemple

Exemple de réseau :
Réseau de distribution d'eau avec :

- Sommet 1 (source) : $b_1 = +100$ (station de pompage)
- Sommets 2, 3 (transit) : $b_2 = b_3 = 0$ (stations intermédiaires)
- Sommets 4, 5 (puits) : $b_4 = -60$, $b_5 = -40$ (villes à alimenter)
- Arc $(1,2)$: $c_{12} = 2$ /mş, $u_{12} = 70$ mş/h, $l_{12} = 10$ mş/h

Vérification : $100 + 0 + 0 + (-60) + (-40) = 0$ (verifie)

5 Complexité d'un Algorithme

5.1 Définition

Définition

La **complexité temps** d'un algorithme est une fonction du nombre d'opérations élémentaires effectuées dans l'algorithme.
On utilise la notation O (grand O) pour donner un ordre de grandeur de la complexité.

Définition

Un algorithme a une complexité $O(f(n))$ s'il existe des constantes c et n_0 tels que le temps pris par l'algorithme dans le pire des cas est au plus $c \cdot f(n)$ pour $n \geq n_0$.

Note de Cours

Classes de complexité courantes :

Notation	Nom	Exemple
$O(1)$	Constante	Accès à un élément d'un tableau
$O(\log n)$	Logarithmique	Recherche dichotomique
$O(n)$	Linéaire	Parcours d'une liste
$O(n \log n)$	Linéarithmique	Tri fusion, tri rapide
$O(n^2)$	Quadratique	Tri à bulles, parcours de matrice
$O(n^3)$	Cubique	Produit de matrices naïf
$O(2^n)$	Exponentielle	Ensemble des parties, problème du sac à dos
$O(n!)$	Factorielle	Problème du voyageur de commerce (force brute)

À savoir

Algorithmes polynomiaux vs exponentiels :

- **Polynomial** ($O(n^k)$) : considérés comme *efficaces* et *résolubles*
- **Exponentiel** ($O(2^n)$, $O(n!)$) : rapidement *impraticables* pour grandes instances

Pour $n = 50$:

- $O(n^2) = 2\,500$ opérations
- $O(2^n) \approx 10^{15}$ opérations (plusieurs jours de calcul)

6 Problèmes de Base dans les Réseaux

Important

Les trois problèmes fondamentaux d'optimisation dans les réseaux sont :

1. **Problème du plus court chemin**
2. **Problème du flot maximum**
3. **Problème du flot de coût minimum**

6.1 Problème du Plus Court Chemin

6.1.1 Description du problème

Définition

Problème : Il s'agit de trouver la façon la plus économique (en temps, distance, difficulté, coût, etc.) de passer d'un nud d'un réseau à un autre.
Données :

- $G = (V, E)$: graphe orienté
- $s \in V$: nud origine (source)
- $p \in V$: nud destination (puits)
- c_{ij} : coût unitaire pour chaque arc $(i, j) \in E$

Objectif : Trouver un chemin de s à p de coût total minimum.

6.1.2 Formulation linéaire

Définition

Variables de décision :

$$x_{ij} = \begin{cases} 1 & \text{si l'arc } (i,j) \text{ est choisi dans le chemin} \\ 0 & \text{sinon} \end{cases}$$

Contraintes (conservation du flot) :

$$\sum_{j \in V} x_{ij} - \sum_{j \in V} x_{ji} = \begin{cases} +1 & \text{si } i = s \text{ (origine)} \\ -1 & \text{si } i = p \text{ (destination)} \\ 0 & \text{sinon (nud de transit)} \end{cases}$$

Fonction objectif :

$$\min \sum_{(i,j) \in E} c_{ij} x_{ij}$$

Note de Cours

Interprétation des contraintes :

- Au nud source s : on sort (+1), on n'entre pas
- Au nud destination p : on entre (-1), on ne sort pas
- Aux nuds de transit : ce qui entre = ce qui sort (conservation)

Ces contraintes garantissent qu'on construit bien un chemin de s à p.

6.1.3 Remarques importantes

Important

1. Circuits négatifs :
L'objectif est **infini** (ou non borné) en présence de circuits négatifs.
Un circuit négatif est un cycle dont la somme des coûts des arcs est négative. On pourrait le parcourir indéfiniment pour réduire le coût à $-\infty$.
2. Relaxation linéaire :
La relaxation linéaire du problème donne une solution **entière**.
Cela signifie qu'on peut résoudre le problème en autorisant $x_{ij} \in [0,1]$ (programmation linéaire), et la solution optimale aura automatiquement des valeurs entières ($x_{ij} \in \{0,1\}$).
Raison : La matrice des contraintes est la matrice d'incidence du graphe, qui est *totalement unimodulaire*.

6.1.4 Algorithmes de résolution

Algorithme

Algorithme de Bellman :

- Contexte : Graphe sans circuits (DAG - Directed Acyclic Graph)
- Complexité : $O(|V| + |E|)$
- Principe : Traitement des sommets par ordre topologique

Algorithme de Dijkstra :

- Contexte : Poids (coûts) positifs ou nuls
- Complexité : $O(|E| + |V| \log |V|)$ avec tas de Fibonacci
- Principe : Sélection gloutonne du sommet de distance minimale

Algorithme de Bellman-Ford :

- Contexte : Cas général (poids négatifs autorisés)
- Complexité : $O(|V| \cdot |E|)$
- Principe : Relaxation itérative de toutes les arêtes
- Bonus : Détecte les circuits négatifs

Exemple

Exemple numérique :
Graphe avec 4 sommets, source = 1, destination = 4.
Arcs et coûts : $(1, 2) : 3$, $(1, 3) : 5$, $(2, 3) : 1$, $(2, 4) : 7$, $(3, 4) : 2$
Plus court chemin : $1 \to 2 \to 3 \to 4$
Coût total : $3 + 1 + 2 = 6$
Autres chemins :

- $1 \to 2 \to 4$: coût $= 3 + 7 = 10$
- $1 \to 3 \to 4$: coût $= 5 + 2 = 7$

6.2 Problème du Flot Maximum

6.2.1 Description du problème

Définition

Problème : Il s'agit d'envoyer la plus grande valeur de flot (quantité, volume, usagers, etc.) à travers un réseau entre deux points en tenant compte d'une capacité limitative.
Données :

- $G = (V, E)$: graphe orienté
- $s \in V$: nud source (origine du flot)
- $t \in V$: nud puits (destination du flot)
- u_{ij} : capacité de l'arc (i, j) avec $0 \leq u_{ij} \leq U$

Objectif : Maximiser le flot envoyé de s à t.

6.2.2 Formulation linéaire

Définition

Variables de décision :

$$x_{ij} = \text{valeur du flot sur l'arc } (i, j)$$

Contraintes :

1. *Conservation du flot aux nuds :*

$$\sum_{j \in V} x_{ij} - \sum_{j \in V} x_{ji} = \begin{cases} +v & \text{si } i = s \text{ (source)} \\ -v & \text{si } i = t \text{ (puits)} \\ 0 & \text{sinon (nuds intermédiaires)} \end{cases}$$

2. *Contraintes de capacité :*

$$0 \leq x_{ij} \leq u_{ij}, \quad \forall (i, j) \in E$$

Fonction objectif :

$$\max \ v$$

où v est la valeur du flot total (quantité sortant de s = quantité entrant dans t).

Note de Cours

Interprétation physique :
Imaginez un réseau de tuyaux :

- La source s injecte v unités de liquide
- Le puits t reçoit v unités de liquide
- Chaque tuyau (arc) a une capacité maximale u_{ij}
- Le liquide est conservé : ce qui entre dans un nud intermédiaire en ressort

On cherche à maximiser v (débit total).

6.2.3 Algorithmes de résolution

Algorithme

Algorithme de Ford-Fulkerson :

- Principe : Recherche itérative de chemins augmentants
- Complexité : $O(|E| \cdot f^*)$ où f^* est la valeur du flot maximum
- Propriété : Peut ne pas converger si les capacités sont irrationnelles

Algorithme d'Edmonds-Karp :

- Principe : Variante de Ford-Fulkerson utilisant BFS pour trouver les chemins
- Complexité : $O(|V| \cdot |E|^2)$
- Avantage : Convergence garantie en temps polynomial

Algorithme Push-Relabel :

- Principe : Approche locale par préflot (peut violer temporairement la conservation)
- Complexité : $O(|V|^2 \cdot |E|)$ version de base, $O(|V|^3)$ améliorée
- Avantage : Efficace en pratique, parallélisable

À savoir

Théorème Flot-Max/Coupe-Min :
La valeur du flot maximum de s à t est égale à la capacité de la coupe minimum séparant s et t.

$$\max\{\text{valeur du flot}\} = \min\{\text{capacité d'une coupe}\}$$

Une *coupe* (S, T) sépare s (dans S) et t (dans T), et sa capacité est :

$$\text{cap}(S, T) = \sum_{i \in S, j \in T, (i,j) \in E} u_{ij}$$

Exemple

Exemple numérique :
Réseau avec source $= 1$, puits $= 4$.
Capacités : $(1,2) : 10$, $(1,3) : 5$, $(2,3) : 15$, $(2,4) : 10$, $(3,4) : 10$
Flot maximum possible : $v = 15$
Répartition :

- Arc $(1,2)$: flot $= 10$ (saturé)
- Arc $(1,3)$: flot $= 5$ (saturé)
- Arc $(2,3)$: flot $= 0$
- Arc $(2,4)$: flot $= 10$ (saturé)
- Arc $(3,4)$: flot $= 5$

Vérification : sortie de $1 = 10 + 5 = 15$, entrée dans $4 = 10 + 5 = 15$ (verifie)

6.3 Problème du Flot de Coût Minimum

6.3.1 Description du problème

Définition

Problème : Il s'agit d'envoyer du flot à travers un réseau entre plusieurs points en tenant compte de capacités limitatives et en minimisant le coût global de circulation.
Données :

- $G = (V, E)$: graphe orienté
- Plusieurs nuds sources et destinations
- b_i : demande ($b_i < 0$) ou offre ($b_i > 0$) au nud i
- $b_i = 0$ aux nuds intermédiaires
- l_{ij} : capacité inférieure avec $0 \leq l_{ij} \leq L$
- u_{ij} : capacité supérieure avec $0 \leq u_{ij} \leq U$
- c_{ij} : coût unitaire par arc (i, j)

Objectif : Satisfaire toutes les demandes au coût minimum.

6.3.2 Formulation linéaire

Définition

Variables de décision :

$$x_{ij} = \text{valeur du flot sur l'arc } (i, j)$$

Contraintes :

1. *Conservation du flot :*

$$\sum_{j \in V} x_{ij} - \sum_{j \in V} x_{ji} = b_i, \quad \forall i \in V$$

2. *Contraintes de capacité :*

$$l_{ij} \leq x_{ij} \leq u_{ij}, \quad \forall (i, j) \in E$$

Fonction objectif :

$$\min \sum_{(i,j) \in E} c_{ij} x_{ij}$$

Note de Cours

Condition de faisabilité :
Pour qu'une solution existe, il faut que :

$$\sum_{i \in V} b_i = 0$$

C'est-à-dire que la somme des offres égale la somme des demandes (en valeur absolue).

Important

Généralisation des problèmes précédents :
Le problème du flot de coût minimum **généralise** :

- Le problème du plus court chemin (avec $b_s = 1$, $b_t = -1$, autres $= 0$)
- Le problème du flot maximum (en ajoutant un arc de retour (t, s) de capacité infinie et coût -1)

6.3.3 Algorithmes de résolution

Algorithme

Principaux algorithmes :

1. **Algorithme du Simplexe sur Réseau :**
   - Variante spécialisée du simplexe pour les réseaux
   - Très efficace en pratique
   - Complexité polynomiale dans certains cas
2. **Algorithme des Chemins Augmentants :**
   - Extension de l'algorithme de Ford-Fulkerson
   - Recherche de chemins augmentants de coût négatif
3. **Algorithme Primal-Dual :**
   - Maintient simultanément une solution primale et duale
   - Converge vers l'optimum primal et dual
4. **Algorithme du Scaling de Capacité :**
   - Traite les arcs par ordre de capacité
   - Complexité polynomiale forte

Exemple

Exemple : Problème de transport
3 usines (offres) et 2 entrepôts (demandes).
Offres :

- Usine 1 : $b_1 = +50$
- Usine 2 : $b_2 = +70$
- Usine 3 : $b_3 = +30$

Demandes :

- Entrepôt A : $b_A = -80$
- Entrepôt B : $b_B = -70$

Coûts de transport (en /unité) :

	Entrepôt A	Entrepôt B
Usine 1	8	6
Usine 2	10	7
Usine 3	5	9

Vérification : $50 + 70 + 30 = 150 = 80 + 70$ (verifie)
Objectif : Minimiser le coût total de transport tout en satisfaisant toutes les demandes.

7 Connexité dans les Réseaux de Télécommunications

7.1 Problèmes généraux sur la connexité

Définition

Les problèmes de connexité étudient :
1. Chaînes/chemins k-arêtes (ou sommets) disjointes : Chercher k chaînes entre deux sommets qui ne partagent aucune arête (ou aucun sommet).
2. Bornes sur chemins et cycles : Étudier les propriétés des chemins et cycles dans un graphe.
3. Point d'articulation : Un sommet v est un point d'articulation si sa suppression augmente le nombre de composantes connexes : $(G - v) \nearrow$ composantes.
4. Ensemble d'articulation : Un ensemble $W \subset V$ est un ensemble d'articulation si $(G \setminus W) \nearrow$ composantes.

Note de Cours

Importance en télécommunications :
La connexité est cruciale pour la **robustesse** et la **fiabilité** des réseaux :

- Résister aux pannes de câbles ou d'équipements
- Garantir la continuité de service
- Permettre le reroutage du trafic

7.2 Architecture des Réseaux de Télécommunications

7.2.1 Les trois niveaux de traitement

Définition

Les réseaux de télécommunications sont structurés en trois niveaux :

1. **Longue distance :**
   - Connection ville à ville
   - Nuds "barrière" (backbone nodes)
   - Infrastructure nationale/internationale
2. **Moyenne distance :**
   - Centres d'échange à l'intérieur des villes
   - Connections barrières $\leftrightarrow$ centres
   - Clusters de clients
3. **Faible distance :**
   - Accès locaux
   - Clients dans des clusters
   - Liaison au centre d'échange local

À savoir

À chaque niveau correspond un **critère d'installation différent** :

- Longue distance : robustesse maximale
- Moyenne distance : équilibre coût/fiabilité
- Faible distance : coût minimal

7.3 Problèmes de Longue Distance

7.3.1 Le long terme (> 10 années)

Définition

Contexte :

- Durée : supérieure à 10 années
- Seule la **topologie** est traitée
- La demande n'est pas assez fiable pour le dimensionnement détaillé

Objectif principal :
Déterminer l'ensemble des câbles connectant tous les nuds sous certaines contraintes de sécurité et au moindre coût.

Note de Cours

Impact de la technologie :
Le développement de la technologie de la **fibre optique** a permis :

- L'utilisation de composants de faibles coûts
- Une capacité presque illimitée
- Trafic très élevé possible

Conséquence : Deux problèmes majeurs émergent :

1. **Économique** : Minimiser les coûts de construction
2. **Sécuritaire** : Restauration (ou prévision) des liens en cas de panne

7.3.2 Le moyen terme (< 5 années)

Définition

Contexte :

- Durée : inférieure à 5 années
- **Dimensionnement** des capacités

Entrée :

- Une matrice de demandes
- La topologie actuelle du réseau

Sortie :

- Réaliser le routage en fonction de la capacité des câbles
- L'ajout d'arêtes est autorisé

7.4 Solutions pour les Longues Distances

7.4.1 Solution de base : les arbres

Définition

Avantages des arbres :

- **Connexité assurée** : tous les nuds sont connectés
- **Moindre coût** : nombre minimum d'arêtes ($|V| - 1$)
- **Construction facile** : algorithmes efficaces (Kruskal, Prim)

Algorithmes de construction :

- Algorithme de Kruskal : $O(|E| \log |E|)$
- Algorithme de Prim : $O(|E| \log |V|)$

7.4.2 Modèle linéaire pour l'arbre de poids minimum

Définition

Variables de décision :

$$x_e = \begin{cases} 1 & \text{si l'arête } e \text{ est choisie} \\ 0 & \text{sinon} \end{cases}$$

Contraintes :

1. *Nombre d'arêtes :*

$$\sum_{e \in E} x_e = |V| - 1$$

2. *Connexité (contraintes de coupe) :*

$$\sum_{e \in \delta(W)} x_e \geq 1, \quad \forall W \subset V, W \neq \emptyset$$

Fonction objectif :

$$\min \sum_{e \in E} c_e x_e$$

Note de Cours

Interprétation des contraintes de coupe :
Pour tout sous-ensemble non vide W de sommets, il doit y avoir au moins une arête reliant W à $V \setminus W$. Cela garantit que le graphe est connexe.
Le nombre de contraintes de coupe est exponentiel ($2^{|V|} - 2$), mais on peut les ajouter dynamiquement (génération de coupes).

Important

Problème des arbres :
Une arête supprimée (câble coupé) $\Rightarrow$ perte de connexité $\Rightarrow$ au moins deux composantes connexes (réseau coupé !).
Solution : Rechercher des topologies plus robustes **réseaux k-connexes**

7.5 Réseaux k-Connexes

7.5.1 Définition

Définition

Un réseau **k-connexe** a la propriété suivante :
Possibilité pour un réseau de résister à la destruction d'au plus $k-1$ câbles entre deux points.
Autrement dit :

- Il existe k chaînes arêtes-disjointes entre toute paire de sommets
- La suppression de moins de k arêtes ne déconnecte pas le réseau

À savoir

Niveaux de robustesse :

- 1-connexe : connexe simple (arbre suffisant)
- 2-connexe : résiste à la panne d'un câble
- k-connexe : résiste à la panne de $k-1$ câbles

Plus k est grand, plus le réseau est robuste, mais plus il est coûteux.

7.5.2 Types de nuds selon les requêtes

Définition

Chaque nud j est caractérisé par une **requête** $r_j \in \{0, 1, 2\}$ indiquant le niveau de fiabilité souhaité.
Entre deux nuds i et j, il faut $\min\{r_i, r_j\}$ chaînes disjointes.
Trois types de nuds :

1. **Nuds spéciaux** ($r_i = 2$) **:**
   - Haut degré de sûreté nécessaire
   - Centres critiques, hôpitaux, services d'urgence
   - Requièrent 2 chemins disjoints avec tout autre nud spécial
2. **Nuds ordinaires** ($r_i = 1$) **:**
   - Doivent simplement être connectés
   - Services standards
3. **Nuds optionnels** ($r_i = 0$) **:**
   - Peuvent ne pas être dans le réseau final
   - Connexion si économiquement viable

7.5.3 Contraintes de robustesse

Important

Contraintes principales :
1. Résistance aux pannes de liens :
La destruction d'un lien ne doit pas déconnecter le réseau entre deux nuds spéciaux.
$\Rightarrow$ Au moins **deux chaînes arêtes-disjointes** entre nuds spéciaux.
2. Résistance aux pannes de nuds :
La destruction d'un nud ne doit pas déconnecter le réseau.
$\Rightarrow$ Au moins **deux chaînes sommets-disjointes** entre toute paire de nuds.

7.6 Cas Particuliers et Complexité

Définition

Soit $G = (V, E)$ un graphe. Selon les valeurs de r_i, on obtient différents problèmes :

1. $r_i = 1, \forall i \in V$

   $\Rightarrow$ **Arbre couvrant** (Minimum Spanning Tree)

   Complexité : *Polynomial* ($O(|E| \log |V|)$)

2. $r_s = r_t = 1$, $r_i = 0$ pour tous les autres

   $\Rightarrow$ **Plus court chemin** de s à t

   Complexité : *Polynomial* ($O(|E| + |V| \log |V|)$)

3. $r_s = r_t = k$ (avec $k \geq 2$), $r_i = 0$ pour tous les autres

   $\Rightarrow$ **k plus courts chemins arêtes-disjoints** de s à t

   Complexité : *Polynomial* ($O(k \cdot |E| \log |V|)$)

4. $r_i \in \{0, 1\}, \forall i \in V$

   $\Rightarrow$ **Arbre de Steiner**

   Complexité : *NP-difficile*

5. $r_i = 2, \forall i \in V$

   $\Rightarrow$ **Réseau 2-connexe de coût minimum**

   Complexité : *NP-difficile*

Note de Cours

Remarques sur la complexité :

- Les problèmes avec connectivité simple (arbres, chemins) sont **polynomiaux**
- Les problèmes avec choix de sommets (Steiner) ou double connectivité sont **NP-difficiles**
- Pour les problèmes NP-difficiles, on utilise :
  - Des heuristiques
  - Des algorithmes d'approximation
  - Des méthodes exactes (Branch & Cut) pour instances petites/moyennes

7.7 Approche de Résolution : Réseau 2-Connexe

7.7.1 Modèle linéaire entier pour $r_i = 2, \forall i \in V$

Définition

Variables de décision :

$$x_e = \begin{cases} 1 & \text{si l'arête } e \text{ est choisie} \\ 0 & \text{sinon} \end{cases}$$

Contraintes (coupes 2-connexes) :

$$\sum_{e \in \delta(W)} x_e \geq 2, \quad \forall W \subset V, W \neq \emptyset, W \neq V$$

Fonction objectif :

$$\min \sum_{e \in E} c_e x_e$$

Note de Cours

Interprétation :
Pour tout sous-ensemble propre W de sommets, il doit y avoir au moins **deux arêtes** reliant W à $V \setminus W$.
Cela garantit qu'on ne peut pas déconnecter le graphe en supprimant une seule arête.

7.7.2 Méthode de résolution : Branch & Cut

Algorithme

Relaxation linéaire + Branch & Cut :
Étape 1 : Relaxation

- On relaxe $x_e \in \{0, 1\}$ en $x_e \in [0, 1]$
- On résout le programme linéaire

Étape 2 : Génération de coupes

- Si la solution n'est pas entière, on cherche des contraintes violées
- On ajoute dynamiquement les contraintes de coupe nécessaires
- Algorithme de séparation : recherche de coupe minimale

Étape 3 : Branchement

- Si la solution n'est toujours pas entière, on branche
- Exploration de l'arbre de recherche (Branch & Bound)

À savoir

Efficacité en pratique :
Bien que NP-difficile en théorie, le Branch & Cut résout efficacement des instances de plusieurs centaines de nuds grâce à :

- La structure particulière du problème
- Les techniques de prétraitement
- Les heuristiques de recherche intelligentes

7.8 Théorème de Menger (1927)

7.8.1 Énoncé pour les réseaux (version arcs)

Définition

Soit $R = (V, E, c)$ un réseau avec :

- c : capacité unitaire sur les arcs
- $s \in V$: source
- $t \in V$: puits

Théorème de Menger (version flot-coupe) :
(a) La valeur d'un flot maximum dans R est égale au nombre maximum de st-chemins arcs-disjoints.
(b) La capacité d'une coupe minimum dans R est égale au nombre minimum d'arcs dont la suppression détruit tous les st-chemins de R.

7.8.2 Énoncé pour les graphes non orientés

Définition

Théorème de Menger (version graphes) :
Soit $G = (V, E)$ un graphe non orienté avec deux sommets non adjacents s et t, et soit k un entier.
Il existe k chaînes de s à t **sommets-disjointes** (resp. **arêtes-disjointes**) si et seulement si t reste connecté à s après suppression de $k - 1$ sommets différents de s et t (resp. arêtes) quelconques.

Note de Cours

Interprétation :
Le nombre maximum de chemins disjoints entre s et t est égal au nombre minimum d'arêtes (ou sommets) qu'il faut supprimer pour séparer s de t.
C'est une généralisation du théorème Flot-Max/Coupe-Min.

7.8.3 Conséquence : Théorème de Whitney (1932)

Important

Caractérisation de la k-connexité :
Un graphe non orienté G ayant au moins 2 sommets est **k-arête-connexe** si et seulement si pour chaque paire $s, t \in V(G)$ avec $s \neq t$, il existe k chaînes de s à t arêtes-disjointes.
Un graphe non orienté G ayant au moins $k+1$ sommets est **k-connexe** (ou k-sommet-connexe) si et seulement si pour chaque paire $s, t \in V(G)$ avec $s \neq t$, il existe k chaînes de s à t sommets-disjointes.

À savoir

Application pratique :
Pour vérifier qu'un réseau est 2-connexe :

- Vérifier que pour toute paire de nuds, il existe 2 chemins sommet-disjoints
- Équivalent : vérifier qu'il n'existe aucun point d'articulation
- Équivalent : vérifier que la suppression de tout nud laisse le graphe connexe

7.9 Exemple d'Application : Réseau Belge Belgacom

7.9.1 Contexte

Exemple

Problème initial :
Le réseau Belgacom connectait 52 centres via un **cycle hamiltonien** (un cycle passant par tous les sommets exactement une fois).
Problème rencontré :
Une panne sur ce cycle ⇒ Reroutage ⇒ Nouvelle route chargée en arêtes !!
Le trafic devait emprunter un très long détour, surchargeant certains liens.

7.9.2 Solution proposée

Définition

Idée : Rechercher des topologies **2-arêtes connexes** avec des **cycles bornés**.
Contraintes :

- Cycles de longueur 3 à 6 arêtes
- L'union de ces cycles doit être couvrante (tous les sommets inclus)
- Minimiser le coût total

Avantages :

- Robustesse : en cas de panne, reroutage local dans un petit cycle
- Efficacité : les détours restent courts
- Modularité : structure en cycles facilite la maintenance

Note de Cours

Référence :
Projet de recherche détaillé disponible sur :
`https://mathopt.be/comex.ulb.ac.be/`
Ce projet illustre l'application concrète des concepts de connexité et d'optimisation de réseaux à un problème réel de grande échelle.

À savoir

Leçon générale :
Dans la conception de réseaux robustes, il ne suffit pas d'assurer la 2-connexité globale. Il faut aussi considérer la **longueur des chemins alternatifs** pour garantir une bonne qualité de service même en cas de panne.

8 Résumé et Points Essentiels

8.1 Structures de graphes

À savoir

Concepts fondamentaux :

- **Graphe** : structure (V, E) avec sommets et arêtes/arcs
- **Matrice d'adjacence** : représente les connexions entre sommets
- **Matrice d'incidence** : relie sommets et arêtes/arcs
- **Degré** : nombre d'arêtes incidentes à un sommet
- **Chaîne/Chemin** : suite alternée de sommets et arêtes/arcs
- **Cycle/Circuit** : chaîne/chemin fermé
- **Connexité** : existence de chaînes entre tous sommets
- **Arbre** : graphe connexe sans cycle

8.2 Problèmes d'optimisation

Important

Trois problèmes fondamentaux :

1. **Plus Court Chemin :**
   - Minimiser le coût total d'un chemin de s à t
   - Algorithmes : Bellman, Dijkstra, Bellman-Ford
   - Complexité polynomiale
2. **Flot Maximum :**
   - Maximiser le flot envoyé de s à t
   - Algorithmes : Ford-Fulkerson, Edmonds-Karp, Push-Relabel
   - Théorème Flot-Max = Coupe-Min
3. **Flot de Coût Minimum :**
   - Minimiser le coût total de circulation du flot
   - Généralise les deux problèmes précédents
   - Algorithme du simplexe sur réseau

8.3 Connexité et robustesse

À savoir

Niveaux de connexité :

- 1-connexe : connexe (arbre suffisant)
- 2-connexe : résiste à une panne d'arête
- k-connexe : résiste à $k - 1$ pannes

Théorèmes clés :

- Théorème de Menger : caractérisation par chemins disjoints
- Théorème de Whitney : caractérisation de la k-connexité

8.4 Applications pratiques

Note de Cours

Domaines d'application :

- Télécommunications : topologie, routage, fiabilité
- Transports : logistique, livraisons, trafic
- Énergie : distribution électrique, réseau de gaz
- Informatique : réseaux, flux de données, internet

Critères de conception :

1. Minimiser les coûts
2. Garantir la robustesse
3. Optimiser les performances
4. Faciliter la maintenance

9 Conclusion

Important

L'optimisation dans les réseaux est un domaine riche et fondamental de la recherche opérationnelle, avec des applications concrètes dans de nombreux secteurs.
Points clés à retenir :

1. Les **graphes** sont des modèles puissants pour représenter des réseaux
2. Les **problèmes classiques** (plus court chemin, flot maximum, flot de coût minimum) ont des algorithmes efficaces bien établis
3. La **connexité** est cruciale pour la robustesse des réseaux
4. Le **compromis coût-fiabilité** guide la conception de réseaux réels
5. Les **méthodes exactes et heuristiques** permettent de résoudre des problèmes de grande taille

À savoir

Pour approfondir :

- Étudier les algorithmes en détail (TD)
- Pratiquer sur des exemples concrets
- Explorer les variantes et extensions des problèmes de base
- S'intéresser aux applications dans votre domaine d'intérêt
- Consulter les ressources du livre blanc de la Roadef

Note de Cours

Perspectives :
Les défis actuels et futurs incluent :

- Réseaux dynamiques et adaptatifs
- Optimisation multi-objectifs
- Prise en compte de l'incertitude
- Réseaux intelligents (smart grids, IoT)
- Optimisation en temps réel
- Intelligence artificielle et apprentissage pour l'optimisation

Bibliographie

1. Association Roadef (2019). *Le livre blanc de la recherche opérationnelle.* `http://www.roadef.org/roadef-livre-blanc`

2. Applegate, D. L., Bixby, R. E., Chvátal, V., Cook, W. J. (2006). *The Traveling Salesman Problem.* Chapitre 1.

3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. (1990). *Introduction to Algorithms.* MIT Press.

4. Simplot-Ryl, D., Fleury, E. (2009). *Réseaux de capteurs - Théorie et modélisation.* Éditions Lavoisier.

5. Ahuja, R. K., Magnanti, T. L., Orlin, J. B. (1993). *Network Flows: Theory, Algorithms, and Applications.* Prentice Hall.

6. Schrijver, A. (2003). *Combinatorial Optimization: Polyhedra and Efficiency.* Springer.

Fin de la partie principale

Annexes : Compléments de Cours

A. Algorithmes Détaillés

A.1 Algorithme de Dijkstra (Plus Court Chemin)

Algorithme

Données : Graphe $G = (V, E)$, source s, poids positifs $c_{ij} \geq 0$
Initialisation :

- $d[s] \leftarrow 0$ (distance de s à s)
- $d[v] \leftarrow +\infty$ pour tout $v \in V \setminus \{s\}$
- $S \leftarrow \emptyset$ (ensemble des sommets traités)
- $Q \leftarrow V$ (file de priorité)

Tant que $Q \neq \emptyset$:

1. Extraire u de Q avec $d[u]$ minimal
2. Ajouter u à S
3. **Pour chaque** voisin v de u :
   - Si $d[u] + c_{uv} < d[v]$:
     - $d[v] \leftarrow d[u] + c_{uv}$
     - $\text{pred}[v] \leftarrow u$

Retourner : tableau d des distances et pred des prédécesseurs

Note de Cours

Complexité :

- Avec tableau simple : $O(|V|^2)$
- Avec tas binaire : $O((|V| + |E|) \log |V|)$
- Avec tas de Fibonacci : $O(|E| + |V| \log |V|)$

Invariant : À chaque itération, pour tout sommet $u \in S$, $d[u]$ est la distance minimale de s à u.

A.2 Algorithme de Bellman-Ford

Algorithme

Données : Graphe $G = (V, E)$, source s, poids c_{ij} (peuvent être négatifs)
Initialisation :

- $d[s] \leftarrow 0$
- $d[v] \leftarrow +\infty$ pour tout $v \in V \setminus \{s\}$

Répéter $|V| - 1$ fois :

- **Pour chaque** arc $(u, v) \in E$:
  - Si $d[u] + c_{uv} < d[v]$:
    - $d[v] \leftarrow d[u] + c_{uv}$
    - $\text{pred}[v] \leftarrow u$

Détection de circuit négatif :

- **Pour chaque** arc $(u, v) \in E$:
  - Si $d[u] + c_{uv} < d[v]$:
    - **Retourner** "Circuit négatif détecté"

Retourner : tableau d des distances et pred des prédécesseurs

Exemple

Exemple avec circuit négatif :
Graphe : $(1, 2) : -1$, $(2, 3) : -2$, $(3, 1) : -3$
Somme du cycle : $-1 + (-2) + (-3) = -6 < 0$
L'algorithme détecte ce circuit et s'arrête.

A.3 Algorithme de Ford-Fulkerson

Algorithme

Données : Réseau $G = (V, E)$, source s, puits t, capacités u_{ij}
Initialisation :

- $f_{ij} \leftarrow 0$ pour tout $(i, j) \in E$ (flot initial nul)

Tant que il existe un chemin augmentant de s à t dans le graphe résiduel :

1. Trouver un chemin augmentant P de s à t
2. Calculer la capacité résiduelle du chemin :

$$\delta = \min\{u_{ij} - f_{ij} : (i, j) \in P\}$$

3. **Pour chaque** arc (i, j) du chemin P :
   - $f_{ij} \leftarrow f_{ij} + \delta$ (augmenter le flot)
   - $f_{ji} \leftarrow f_{ji} - \delta$ (arc retour)

Retourner : flot f et sa valeur $\sum_{(s,v)\in E} f_{sv}$

Note de Cours

Graphe résiduel :
Pour chaque arc (i, j) avec flot f_{ij} et capacité u_{ij} :

- Capacité résiduelle avant : $r_{ij} = u_{ij} - f_{ij}$
- Capacité résiduelle arrière : $r_{ji} = f_{ij}$

Un chemin est augmentant s'il existe un chemin de s à t dans le graphe résiduel avec $r_{ij} > 0$ sur tous les arcs.

A.4 Algorithme de Kruskal (Arbre Couvrant Minimal)

Algorithme

Données : Graphe $G = (V, E)$, poids c_e pour chaque arête e
Initialisation :

- $T \leftarrow \emptyset$ (ensemble des arêtes de l'arbre)
- Trier les arêtes par ordre croissant de poids
- Initialiser une structure Union-Find pour V

Pour chaque arête $e = (u, v)$ dans l'ordre croissant :

- Si u et v sont dans des composantes différentes :
  - Ajouter e à T
  - Unir les composantes de u et v
- Si $|T| = |V| - 1$: **Arrêter**

Retourner : arbre T

Exemple

Exemple :
Graphe avec arêtes et poids :

- $(1, 2) : 1$, $(2, 3) : 2$, $(1, 3) : 4$, $(3, 4) : 3$, $(2, 4) : 5$

Étapes :

1. Ajouter $(1, 2)$: coût $= 1$, $T = \{(1, 2)\}$
2. Ajouter $(2, 3)$: coût $= 2$, $T = \{(1, 2), (2, 3)\}$
3. Ignorer $(1, 3)$: créerait un cycle
4. Ajouter $(3, 4)$: coût $= 3$, $T = \{(1, 2), (2, 3), (3, 4)\}$

Arbre final : coût total $= 1 + 2 + 3 = 6$

B. Exercices et Problèmes

Exercice 1 : Plus Court Chemin

Exemple

Soit le graphe orienté suivant avec les coûts :
Sommets : $\{A, B, C, D, E\}$
Arcs et coûts :

- $(A, B) : 4$, $(A, C) : 2$
- $(B, C) : 1$, $(B, D) : 5$
- $(C, D) : 8$, $(C, E) : 10$
- $(D, E) : 2$

Questions :

1. Appliquer l'algorithme de Dijkstra pour trouver le plus court chemin de A à E
2. Donner la distance minimale et le chemin correspondant

Solution :
Initialisation :

- $d[A] = 0$
- $d[B] = d[C] = d[D] = d[E] = +\infty$

Itération 1 : Traiter A

- $d[B] = \min(\infty, 0 + 4) = 4$
- $d[C] = \min(\infty, 0 + 2) = 2$

Itération 2 : Traiter C (distance minimale $= 2$)

- $d[B] = \min(4, 2 + 1) = 3$
- $d[D] = \min(\infty, 2 + 8) = 10$
- $d[E] = \min(\infty, 2 + 10) = 12$

Itération 3 : Traiter B (distance $= 3$)

- $d[D] = \min(10, 3 + 5) = 8$

Itération 4 : Traiter D (distance $= 8$)

- $d[E] = \min(12, 8 + 2) = 10$

Résultat :

- Plus court chemin : $A \to C \to B \to D \to E$
- Distance minimale : 10

Exercice 2 : Flot Maximum

Exemple

Soit le réseau suivant :
Sommets : source s, puits t, nuds intermédiaires $\{1, 2, 3\}$
Capacités :

- $(s, 1) : 10$, $(s, 2) : 8$
- $(1, 2) : 2$, $(1, 3) : 9$
- $(2, 3) : 7$
- $(3, t) : 10$, $(1, t) : 5$

Question : Calculer le flot maximum de s à t.
Solution par Ford-Fulkerson :
Chemin augmentant 1 : $s \to 1 \to 3 \to t$

- Capacité résiduelle : $\min(10, 9, 10) = 9$
- Flot ajouté : 9

Chemin augmentant 2 : $s \to 2 \to 3 \to t$

- Capacité résiduelle : $\min(8, 7, 10 - 9) = 1$
- Flot ajouté : 1

Chemin augmentant 3 : $s \to 1 \to t$

- Capacité résiduelle : $\min(10 - 9, 5) = 1$
- Flot ajouté : 1

Chemin augmentant 4 : $s \to 2 \to 1 \to t$ (utilise arc retour)

- Capacité résiduelle : $\min(8 - 1, 2, 5 - 1) = 2$
- Flot ajouté : 2

Flot maximum total : $9 + 1 + 1 + 2 = 13$
Vérification par coupe minimale :
Coupe $S = \{s, 1, 2\}$, $T = \{3, t\}$
Capacité : $(1, 3) + (2, 3) = 9 + 7 = 16$ (incorrect)
Coupe $S = \{s\}$, $T = \{1, 2, 3, t\}$
Capacité : $(s, 1) + (s, 2) = 10 + 8 = 18$ (incorrect)
Coupe optimale $S = \{s, 2\}$, $T = \{1, 3, t\}$
Capacité : $(s, 1) + (2, 3) + (2, 1) = 10 + 7 - 4 = 13$ (correct)

C. Propriétés Avancées des Graphes

C.1 Graphes Eulériens et Hamiltoniens

Définition

Graphe Eulérien :
Un graphe possède un **cycle eulérien** (qui passe par chaque arête exactement une fois) si et seulement si :

- Le graphe est connexe
- Tous les sommets ont un degré pair

Un graphe possède une **chaîne eulérienne** s'il a exactement deux sommets de degré impair.
Graphe Hamiltonien :
Un graphe possède un **cycle hamiltonien** (qui passe par chaque sommet exactement une fois) si... il n'existe pas de caractérisation simple !
Le problème de l'existence d'un cycle hamiltonien est NP-complet.

Exemple

Graphe Eulérien :
Le graphe K_4 (complet à 4 sommets) :

- Chaque sommet a degré 3 (impair)
- Il existe une chaîne eulérienne mais pas de cycle eulérien

Le graphe K_5 :

- Chaque sommet a degré 4 (pair)
- Il existe un cycle eulérien

Problème des sept ponts de Königsberg :
Question historique d'Euler : peut-on traverser tous les ponts exactement une fois ?
Réponse : Non, car le graphe correspondant a 4 sommets de degré impair.

C.2 Coloration de Graphes

Définition

Une **k-coloration** d'un graphe est une affectation de k couleurs aux sommets telle que deux sommets adjacents ont des couleurs différentes.
Le **nombre chromatique** $\chi(G)$ est le nombre minimum de couleurs nécessaires pour colorer G.

À savoir

Propriétés :

- $\chi(K_n) = n$ (graphe complet)
- $\chi(G) \leq \Delta(G) + 1$ (théorème de Brooks)
- $\chi(G) = 2$ si et seulement si G est biparti
- Déterminer $\chi(G)$ est un problème NP-difficile

Exemple

Applications :

- Planification : attribution de créneaux horaires sans conflit
- Allocation de fréquences : réseaux de télécommunication
- Allocation de registres : compilation de programmes

D. Tableaux Récapitulatifs

D.1 Complexités des Algorithmes

Note de Cours

Algorithme	**Problème**	**Complexité**
Dijkstra	Plus court chemin (poids ≥ 0)	$O(\|E\| + \|V\| \log \|V\|)$
Bellman-Ford	Plus court chemin (général)	$O(\|V\| \cdot \|E\|)$
Bellman	Plus court chemin (DAG)	$O(\|V\| + \|E\|)$
Floyd-Warshall	Tous les plus courts chemins	$O(\|V\|^3)$
Ford-Fulkerson	Flot maximum	$O(\|E\| \cdot f^*)$
Edmonds-Karp	Flot maximum	$O(\|V\| \cdot \|E\|^2)$
Push-Relabel	Flot maximum	$O(\|V\|^2 \cdot \|E\|)$
Kruskal	Arbre couvrant minimal	$O(\|E\| \log \|E\|)$
Prim	Arbre couvrant minimal	$O(\|E\| \log \|V\|)$
Simplexe réseau	Flot de coût minimum	Polynomial (pratique)

Note : f^* désigne la valeur du flot maximum

D.2 Classes de Complexité des Problèmes

Important

Problème	**Classe**	**Remarque**
Plus court chemin	P	Polynomial
Arbre couvrant minimal	P	Polynomial
Flot maximum	P	Polynomial
Flot de coût minimum	P	Polynomial
Couplage maximum	P	Polynomial
Cycle hamiltonien	NP-complet	Très difficile
Voyageur de commerce	NP-difficile	Optimisation
Arbre de Steiner	NP-difficile	Optimisation
Réseau k-connexe ($k \geq 2$)	NP-difficile	Optimisation
Coloration minimale	NP-difficile	Optimisation

E. Formules et Théorèmes Importants

À savoir

Formule d'Euler pour les graphes planaires :
Pour un graphe planaire connexe :

$$|V| - |E| + |F| = 2$$

où $|F|$ est le nombre de faces (régions).
Lemme des poignées de main :

$$\sum_{v \in V} d(v) = 2|E|$$

Inégalité pour les graphes planaires :
Si G est planaire simple avec $|V| \geq 3$:

$$|E| \leq 3|V| - 6$$

Théorème de Cayley :
Le nombre d'arbres couvrants différents d'un graphe complet K_n est :

$$n^{n-2}$$

Par exemple, K_4 a $4^2 = 16$ arbres couvrants différents.

F. Conseils pour la Résolution de Problèmes

Note de Cours

Méthodologie générale :

1. **Modélisation**
   - Identifier les sommets et les arcs/arêtes
   - Déterminer les poids, capacités, coûts
   - Choisir le type de graphe (orienté/non orienté)
2. **Choix de l'algorithme**
   - Plus court chemin vérifier si poids négatifs
   - Flot identifier source et puits
   - Connexité déterminer le niveau requis
3. **Application**
   - Initialiser correctement les structures
   - Respecter les étapes de l'algorithme
   - Vérifier les invariants à chaque itération
4. **Vérification**
   - Tester sur des cas simples
   - Vérifier les contraintes
   - Comparer avec les bornes théoriques

Important

Pièges courants à éviter :

- Ne pas confondre chaîne et chemin (graphe orienté)
- Oublier de tester l'existence de circuits négatifs
- Mal initialiser les distances à l'infini
- Confondre flot et capacité
- Oublier la conservation du flot aux nuds intermédiaires

10 Structures et Algorithmes dans les Réseaux Sans Fil

Note de Cours

Introduction aux réseaux sans fil :
Ce chapitre traite des problématiques spécifiques aux réseaux sans fil et aux réseaux de capteurs, où les contraintes énergétiques et la nature distribuée des communications imposent des approches particulières d'optimisation.

10.1 Généralités sur les Réseaux Sans Fil

10.1.1 Réseaux multisauts

Définition

Réseau multisauts (Multi-hop network) :
Un réseau où les communications entre deux nuds distants passent par des nuds intermédiaires qui relaient les messages.
Cas particulier : Réseaux de capteurs

- Ce sont des réseaux de communication virtuels à "simplifier"
- L'ensemble des nuds du réseau = l'ensemble des capteurs
- Un capteur est une unité électronique munie d'une batterie
- **Son énergie est limitée** (contrainte majeure)

Important

Caractéristiques d'un capteur :

- Batterie de capacité limitée (durée de vie critique)
- Capacité de calcul limitée
- Mémoire restreinte
- Communication sans fil avec portée limitée
- Capteur de données environnementales (température, humidité, mouvement, etc.)

10.1.2 Structure du réseau virtuel

Définition

Arêtes du réseau :
Deux capteurs i et j sont **virtuellement voisins** selon la portée d'émission de chacun.
Problème :
Chaque nud peut avoir un **très grand nombre de voisins** $\Rightarrow$ beaucoup de communications, perte d'informations et conflits.
Solution :
Nécessité d'ignorer certains liens pour optimiser les communications et économiser l'énergie.

Note de Cours

Notations :
Soit $G = (V, E)$ le réseau :

- V : ensemble des capteurs (nuds)
- E : ensemble des liens de communication potentiels (arêtes virtuelles)
- Pour tout $u \in V$:
  - $N(u)$: ensemble des voisins de u
  - $N[u] = \{u\} \cup N(u)$: voisinage fermé

Le voisinage d'un nud est obtenu par **échange de messages** et dépend de la **distance** entre deux nuds.

10.2 Découverte de Voisinage et Élimination

10.2.1 Filtrage du voisinage

Définition

Filtrage dans le voisinage :
Suppression d'arêtes du graphe $G = (V, E)$ initial virtuel.
$\Rightarrow$ On obtient un graphe partiel $G' = (V, E')$ avec $E' \subset E$.
Objectif principal :
Avoir moins de liens tout en conservant les propriétés essentielles du réseau.

Important

Avantages de l'élimination de liens :

1. **Efficacité des échanges** : moins de messages à gérer
2. **Économie d'énergie** : moins de transmissions
3. **Durée de vie prolongée** : batteries préservées

Contraintes à respecter :

- Minimiser $|E'|$ (nombre d'arêtes)
- G' doit être **connexe**
- G' doit être **planaire**

10.3 Contrôle de Topologie

10.3.1 Types d'algorithmes

Définition

Algorithmes d'affectation de puissances :

1. **Algorithmes centralisés :**
   - Nécessitent la connaissance globale du réseau
   - Un nud central prend toutes les décisions
   - Avantage : solution optimale possible
   - Inconvénient : nécessite communication globale coûteuse
2. **Algorithmes locaux :**
   - Chaque nud prend sa décision à partir d'informations locales
   - Décisions distribuées
   - Avantage : passage à l'échelle, robustesse
   - Inconvénient : peut-être sous-optimal

10.3.2 Types de contrôle de l'énergie

Définition

Deux approches principales :

1. **Organisation de l'affectation de puissances de diffusion**
   - Ajuster la puissance d'émission de chaque nud
   - Compromis entre portée et consommation
2. **Diminution du nombre de retransmissions**
   - Sélection de nuds relais
   - Éviter les retransmissions redondantes

10.4 Algorithmes Centralisés

10.4.1 Algorithme 1 : Arbre couvrant minimum (MST)

Algorithme

Structure recherchée : Arbre couvrant minimum (Minimum Spanning Tree)
Algorithme :

1. Appliquer Prim ou Kruskal sur le graphe $G = (V, E)$
2. Affecter aux nuds une puissance correspondant aux arêtes de cet arbre

Avantages :

- ++ Graphe connexe garanti
- ++ Graphe planaire
- ++ Nombre minimal d'arêtes : $|V| - 1$

Inconvénients :

- – Solution approchée pour la consommation totale
- – Certains nuds peuvent avoir une puissance élevée

10.4.2 Algorithme 2 : BIP (Broadcast Incremental Power)

Algorithme

Broadcast Incremental Power Protocol
Données :

- Graphe $G = (V, E)$
- Racine s (nud source)
- Fonction de coût $C : E \to \mathbb{R}$ (coût des arcs)

Résultat : Un arbre G'
Algorithme :

1. $\forall u \in V$, $p(u) \leftarrow 0$ (initialisation des puissances)
2. Marquer le nud racine s
3. *Un nud marqué appartient à l'arbre BIP*
4. **Tant que** il existe un nud non marqué **faire**
   - Choisir l'arc (u, v) avec u marqué, v non marqué
   - et qui minimise $C(u, v) - p(u)$
   - $p(u) \leftarrow C(u, v)$
   - Marquer v
5. **Fin Tant que**

Note de Cours

Principe de BIP :
L'algorithme exploite le fait qu'augmenter légèrement la puissance d'un nud déjà émetteur coûte moins cher que d'activer un nouveau nud.
À chaque étape, on choisit l'extension la plus économique de l'arbre en cours de construction.

10.4.3 BIP Adapté (Version Bidirectionnelle)

Algorithme

BIP Adapté (pour communications bidirectionnelles)
Données :

- Graphe $G = (V, E)$
- Racine s
- Fonction de coût $C : E \to \mathbb{R}$

Résultat : Un arbre G'
Algorithme :

1. $\forall u \in V$, $p(u) \leftarrow 0$
2. Marquer le nud racine s
3. **Tant que** il existe un nud non marqué **faire**
   - Choisir l'arête (u, v) avec u marqué, v non marqué
   - et qui minimise $a_u + a_v$ où :
     - $a_u = \max\{C(u, v) - p(u), 0\}$
     - $a_v = \max\{C(u, v) - p(v), 0\}$
   - $p(u) \leftarrow p(u) + a_u$
   - $p(v) \leftarrow p(v) + a_v$
   - Marquer v
4. **Fin Tant que**

Note de Cours

Différence avec BIP classique :
Dans BIP adapté, on prend en compte le coût pour **les deux nuds** de l'arête, car dans les réseaux sans fil, la communication est souvent bidirectionnelle.
Les deux nuds doivent avoir une puissance suffisante pour communiquer.

10.5 Algorithmes Locaux

10.5.1 LMST (Local Minimum Spanning Tree)

Algorithme

Arbre couvrant minimum local
Principe : Chaque nud construit un MST local sur son voisinage.
Algorithme :

1. LMST $\leftarrow \emptyset$
2. **Pour** $u \in V$ **faire**
   - $G' \leftarrow \mathrm{MST}(N[u])$
     (Calcul du MST sur le voisinage fermé de u)
     Note : nécessite une connaissance à 2 sauts, car il faut connaître les arêtes entre voisins
   - **Pour** $v \in N_{G'}(u)$ **faire**
     - LMST $\leftarrow$ LMST $\cup \{(u, v)\}$
   - **Fin Pour**
3. **Fin Pour**

Note de Cours

Connaissance requise :
Pour qu'un nud u puisse calculer $\mathrm{MST}(N[u])$, il doit connaître :

- Ses voisins directs (à 1 saut)
- Les arêtes entre ses voisins (à 2 sauts)

Cette information est obtenue par échange de messages de découverte de voisinage.

10.5.2 RNG (Relative Neighborhood Graph)

Définition

Graphe de voisinage relatif
Soit $G = (V, E)$ un graphe avec fonction de poids $p : E \to \mathbb{R}$.
Le graphe de voisinage relatif est $\mathrm{RNG}(G) = (V, E_{\mathrm{RNG}})$ où :

$$E_{\mathrm{RNG}} = \{(u, v) \in E : \nexists w \in N(u) \cap N(v) \text{ tel que } p(u, w) < p(u, v) \text{ et } p(v, w) < p(u, v)\}$$

Interprétation géométrique :
Une arête (u, v) est dans le RNG s'il n'existe pas de nud w qui soit **plus proche** à la fois de u et de v que u et v ne le sont l'un de l'autre.

Important

Propriété d'inclusion :

$$\mathrm{MST}(G) \subseteq \mathrm{LMST}(G) \subseteq \mathrm{RNG}(G)$$

Conséquences :

- Le RNG contient toujours un arbre couvrant
- Le RNG est connexe si G est connexe
- Le LMST est un compromis entre MST (optimal global) et RNG (construction locale)

Exemple

Exemple de RNG :
Considérons trois nuds u, v, w avec distances :

- $p(u, v) = 10$
- $p(u, w) = 6$
- $p(v, w) = 7$

L'arête (u, v) n'est **pas** dans le RNG car :

- $w \in N(u) \cap N(v)$
- $p(u, w) = 6 < 10 = p(u, v)$ ✓
- $p(v, w) = 7 < 10 = p(u, v)$ ✓

Le nud w est "témoin" que (u, v) ne devrait pas être dans le graphe.

10.6 Structuration du Réseau : Organisation des Échanges

10.6.1 Protocoles de routage

Définition

Protocoles de routage dans les réseaux sans fil :

1. **Protocoles proactifs :**
   - Les routes sont établies et maintenues en permanence sur chaque nud
   - Avantage (++) : Un nud connaît la route vers une destination au besoin (pas de délai)
   - Inconvénient (--) : Taille importante des tables de routage, trafic de mise à jour important
2. **Protocoles réactifs :**
   - Les routes sont recherchées à la demande
   - Avantage (++) : Économie de place mémoire et de trafic de contrôle
   - Inconvénient (--) : Chemins potentiellement longs et inondation du réseau par des messages redondants lors de la découverte

Note de Cours

Organisation du réseau en mode réactif :
Pour limiter les inconvénients des protocoles réactifs, on utilise des structures de **"Dominants"** (ou ensembles dominants).
Ces structures permettent de limiter le nombre de nuds participant au routage.

10.7 Dominants et Clustering

10.7.1 Définitions

Définition

Dominant (Dominating Set)
Soit $G = (V, E)$ un graphe.
$D \subset V$ est un **dominant** si $\forall u \in V \setminus D$, $\exists v \in D$ tel que $u \in N(v)$.
Autrement dit : Tout nud est soit dans D, soit voisin d'un nud de D.
Dominant connexe :
Le sous-graphe induit par D est connexe.
Dominant stable (Independent Set) :
Le sous-graphe induit par D est vide d'arêtes (aucun arc entre nuds de D).

Important

Problème d'optimisation :
Trouver un dominant de **cardinalité minimum** est un problème **NP-Complet**.
Conséquence : On utilise des heuristiques et des algorithmes d'approximation.

10.7.2 Clustering (Partitionnement en groupes)

Définition

Topologie par dominant = Clustering
Découpage virtuel du réseau en groupes (clusters).
Structure d'un groupe :

- **1 nud chef** (cluster head) : coordonne le groupe
- **Points relais** (gateways) : assurent la communication inter-groupes
- **Nuds ordinaires** (members) : communiquent via le chef

À savoir

Avantages du clustering :

1. ++ Taille économique des tables de routage
   - Seuls les chefs maintiennent des routes complètes
   - Les membres ne connaissent que leur chef
2. ++ Ressources partagées et stabilité
   - Allocation de ressources au niveau du cluster
   - Moins sensible à la mobilité locale
3. ++ Hiérarchie facilite la gestion
   - Agrégation de données au niveau du chef
   - Communication inter-clusters simplifiée

Inconvénients :

- − Choix de la métrique pour la division en groupes
  - Distance ? Énergie ? Connectivité ?
  - Compromis à trouver selon l'application
- − Surcharge des chefs de cluster
  - Consommation d'énergie plus élevée
  - Nécessité de rotation des rôles

Exemple

Exemple d'application : LEACH
Low-Energy Adaptive Clustering Hierarchy est un protocole célèbre qui :

- Forme des clusters dynamiquement
- Alterne les rôles de chef pour équilibrer l'énergie
- Utilise une communication hiérarchique :
  - Intra-cluster : membres $\rightarrow$ chef
  - Inter-cluster : chef $\rightarrow$ station de base
- Prolonge significativement la durée de vie du réseau

10.8 Algorithmes de Diffusion

Note de Cours

Problématique de la diffusion (Broadcast) :
Dans un réseau sans fil, lorsqu'un nud émet, tous ses voisins reçoivent le message.
Problème : Si tous les nuds retransmettent, on a une **tempête de diffusion** (broadcast storm) :

- Collisions multiples
- Congestion du réseau
- Gaspillage d'énergie

Solution : Sélectionner un sous-ensemble de nuds qui retransmettent (ensemble dominant connexe).

Algorithme

Approches pour la diffusion efficace :

1. **Diffusion basée sur dominant connexe :**
   - Calculer un dominant connexe D
   - Seuls les nuds de D retransmettent
   - Garantit que tous les nuds reçoivent le message
2. **Diffusion probabiliste :**
   - Chaque nud retransmet avec probabilité p
   - Simple mais pas de garantie de couverture
3. **Diffusion basée sur compteur :**
   - Un nud compte combien de fois il reçoit le message
   - Retransmet seulement si le compteur est faible
   - Indique qu'il a peu de voisins qui ont retransmis
4. **Diffusion basée sur la distance :**
   - Retransmettre seulement si on ajoute une "nouvelle" couverture
   - Nécessite connaissance de la position ou des distances

10.9 Synthèse : Réseaux Sans Fil

Important

Points clés à retenir :

1. **Contraintes spécifiques :**
   - Énergie limitée : batteries non rechargeables
   - Communications coûteuses : la radio consomme beaucoup
   - Topologie dynamique : pannes, mobilité
2. **Contrôle de topologie :**
   - Réduire le nombre de liens tout en gardant la connexité
   - MST, LMST, RNG : compromis local/global
   - BIP : minimiser la puissance totale
3. **Organisation du réseau :**
   - Protocoles réactifs vs proactifs
   - Clustering et dominants
   - Hiérarchie pour l'efficacité
4. **Diffusion efficace :**
   - Éviter la tempête de diffusion
   - Sélection intelligente des relais
   - Ensemble dominant connexe

À savoir

Applications pratiques :
Les concepts étudiés s'appliquent à :

- Réseaux de capteurs environnementaux
- Internet des objets (IoT)
- Réseaux ad hoc mobiles (MANET)
- Réseaux de véhicules (VANET)
- Smart cities et bâtiments intelligents

Fin du Cours Complet

Optimisation dans les Réseaux

Réseaux Filaires et Sans Fil

Bon courage pour vos études et examens !

Ce document est une compilation enrichie des cours de F. Bendali-Mailfert
avec exemples, exercices et compléments théoriques
incluant les réseaux filaires, sans fil et les réseaux de capteurs

Version 2025 - Document pédagogique complet et enrichi